MOD -03 - JDK new Features

MOD -03 - JDK new Features

LUMINAR
TECHNOLAB

JDK 5.0 Important Features

Java 5.0 (also known as J2SE 5.0 or Java 1.5) was a major release that introduced several powerful features to enhance performance, readability, and usability. Below are the most important features of Java 5:

1. Enhanced for loop
2. Generics
3. Annotation
4. Autoboxing and Unboxing
5. Enum(Enumerations)
6. Static Imports
7. Varargs (Variable Arguments)
8. Concurrency Utilities (java.util.concurrent package)

Java For-each Loop | Enhanced For Loop

1. The Java for-each loop or enhanced for loop is introduced since J2SE 5.0.
2. It provides an alternative approach to traverse the array or collection in Java.
3. It is mainly used to traverse the array or collection elements.
4. The advantage of the for-each loop is that it eliminates the possibility of bugs and makes the code more readable.
5. It is known as the for-each loop because it traverses each element one by one.

1. The drawback of the enhanced for loop is that it cannot traverse the elements in reverse order.
2. Here, you do not have the option to skip any element because it does not work on an index basis.
3. Moreover, you cannot traverse the odd or even elements only.
4. But, it is recommended to use the Java for-each loop for traversing the elements of array and collection because it makes the code readable.

Syntax

The syntax of Java for-each loop consists of data_type with the variable followed by a colon (:), then array or collection.

```
for(<data_type><variable >: <array | collection>){
//body of for-each loop
}
```

- The Java for-each loop traverses the array or collection until the last element.
- For each element, it stores the element in the variable and executes the body of the for-each loop.

Generics

JDK 5.0 introduced Java Generics

Generics in Java is a feature that allows you to define classes, interfaces, and methods with type parameters.

1. Introduced type safety for collections and other classes.
2. Eliminates the need for explicit type casting.

Why Use Generics?

1. Type Safety: Ensures that only specific types are used in a collection or method.
2. Code Reusability: Allows writing a single method or class that works with multiple types.
3. Eliminates Type Casting: No need for explicit type conversion.
4. Compile-Time Checking: Errors are caught at compile time rather than runtime.

The whole collection framework was re-written to use generics for type-safety with the aim of reducing bugs and adding an extra layer of abstraction over types.

Example scenario for the need for Generics

Let's imagine a scenario where we want to create a list in Java to store Integer.

We write the following:

```
List al = new ArrayList();
al.add(new Integer(1));
Integer i = al.iterator().next();
```

The compiler will complain about the last line. It doesn't know what data type (al.iterator().next()) is returned.

Ie. The compiler will require an explicit casting:

```
Integer i = (Integer) list.iterator.next();
```

There is no guarantee that the return type of the list is an Integer. The defined list(al) could hold any object. We only know that we are retrieving a list by inspecting the context. When looking at types, it can only guarantee that it is an Object and therefore requires an explicit cast to ensure that the type is safe.

This cast can be irritating — we know that the data type in this list is an Integer. The cast is making a mess of our code. It can cause type-related(data type) runtime errors if a programmer makes a mistake with the explicit casting.

It would be much easier if programmers could express their intention to use specific types and the compiler ensured the correctness of such types. This is the idea behind generics.

Let's modify the first line of the previous code snippet:

```
List<Integer> al = new ArrayList<>();
```

By adding the diamond operator <> containing the type, we specify the type(this list to only Integer type) held inside the list. The compiler can execute the type at compile time.

In larger programs, this can add notable robustness and makes the program easier to read.

```
List list = new ArrayList();
list.add("abc");
list.add(new Integer(5)); //OK

for(Object obj : list){
    //type casting leading to ClassCastException at runtime
    String str=(String) obj;
}
```

```
List<String> list = new ArrayList<String>();
list.add("abc");
//list.add(new Integer(5)); //compiler error

for(String str : list){
    //no type casting needed, avoids ClassCastException
}
```

Topics

Java Generic Type

1. Java Generic Class
2. Java Generic Interface
3. Java Generic Method
4. Java Generics Bounded Type Parameters
5. Wildcard in Java

Java Generic Type

Java Generic Type Naming convention helps us understanding code easily and having a naming convention is one of the best practices of Java programming language.

So generics also comes with its own naming conventions. Usually, type parameter names are single, uppercase letters to make it easily distinguishable from java variables.

The most commonly used type parameter names are:

E - Element (used extensively by the Java Collections Framework, for example ArrayList, Set etc.)

K - Key (Used in Map)

V - Value (Used in Map)

N - Number

T - Type

1. Java Generic Class

We can define our own classes with generics type.

A generic type is a class or interface that is parameterized over types.

We use angle brackets (<>) to specify the type parameter.

Before Generics

```java
package genericpackage;

public class GenericsOldStyle {
    private Object t;
    public Object get() {
        return t;
    }
    public void set(Object t) {
        this.t = t;
    }
    public static void main(String args[]){
        GenericsOldStyle type = new GenericsOldStyle();

        type.set("Bini");

        String str = (String) type.get();

        System.out.println(str);
        type.set(20);
        String str = (String) type.get(); //type casting, error prone and can cause
ClassCastException
        System.out.println(str);
    }
}
```

```java
package genericpackage;
public class GenericsType<T> {
    private T t;
    public T get(){
        return this.t;
    }
    public void set(T t1){
        this.t=t1;
    }
    public static void main(String args[]){
        GenericsType<String> type = new GenericsType<>(); // T as String
        type.set("Bini");
        System.out.println(type.get());

        GenericsType<Integer> type2 = new GenericsType<>(); // T as String
        type2.set(10); //valid
        System.out.println(type2.get());
    }
}
```

Above Code Details

1. We don't need to do type-casting and we can remove ClassCastException at runtime.
2. If we don't provide the type at the time of creation, the compiler will produce a warning that "GenericsType is a raw type. "References to generic type GenericsType<T> should be parameterized".
3. When we don't provide the type, the type becomes Object and hence it's allowing both String and Integer objects.
4. But, we should always try to avoid this because we will have to use type casting while working on raw type that can produce runtime errors.

2. Java Generic Interface

Comparable interface is a great example of Generics in interfaces.

```
import java.util.*;

public interface Comparable<T> {
    public int compareTo(T ob);
}
```

1. we can create generic interfaces in java.
2. We can also have multiple type parameters as in Map interface. (type <K,V>)
3. we can provide parameterized value to a parameterized type also,

for example :

new HashMap<String, List<String>>(); is valid.

3. Java Generic Method

- Sometimes we don’t want the whole class to be parameterized, in that case, we can create java generics method.
- Since the constructor is a special kind of method, we can use generics type in constructors too.

The Syntax for Defining Generic Method

```
<type-Parameters> <return_type> <method_name>(parameter list) {
  ..
}
```

- The syntax for a generic method includes a list of type parameters, inside angle brackets, which appears before the method's return type.
- For static generic methods, the type parameter section must appear before the method's return type.

After Generics

Create Generic equals method

Create Generic class GenType.

```
package genericpackage;

class GenType<T> {

    private T t;

    public T get() {
        return this.t;
    }

    public void set(T t1) {
        this.t = t1;
    }
}
```

```java
public class GenericsMethods {
//Generics in method
    public static <T> boolean isEqual(GenType<T> g1, GenType<T> g2){
        return g1.get().equals(g2.get());
    }

//main ()

    public static void main(String args[]){

    // ------------T as String----------------------------
        GenType<String> g1 = new GenType<>();
        g1.set("Bini"); //first object g1

        GenType<String> g2 = new GenType<>();
        g2.set("bini"); //second object g1

        boolean isEqual = GenericsMethods.<String>isEqual(g1, g2);
        System.out.println(isEqual);

        //above statement can be written simply as
        isEqual = GenericsMethods.isEqual(g1, g2);
        System.out.println(isEqual);
```

```java
// ------------T as Integer---------------------------
        GenType<Integer> g3 = new GenType<>();
    g3.set(10); //first object g1

    GenType<Integer> g4 = new GenType<>();
    g4.set(20); //second object g1

    isEqual = GenericsMethods.isEqual(g3, g4);
    System.out.println(isEqual);

// ------------T as Float---------------------------
        GenType<Float> g5 = new GenType<>();
    g5.set(10.5f); //first object g1

    GenType<Float> g6 = new GenType<>();
    g6.set(10.5f); //second object g1

    isEqual = GenericsMethods.isEqual(g5, g6);
    System.out.println(isEqual);
   }
}
```

4. Java Generics Bounded Type Parameters

Suppose we want to restrict the type of objects that can be used in the parameterized type, e.g.in a method that compares two objects and we want to make sure that the accepted objects are Comparables.

To declare a bounded type parameter :

list the type parameter's name, followed by the extends keyword, followed by its upper bound.

```
public static <T extends Comparable<T>> int compare(T t1, T t2){
        return t1.compareTo(t2);
    }
```

The invocation of these methods is similar to unbounded method except that:

- if we will try to use any class that is not Comparable, it will throw compile-time error.
- Bounded type parameters can be used with methods as well as classes and interfaces.
- Java Generics supports multiple bounds also, i.e <T extends A & B & C>. In this case, A can be an interface or class. If A is class then B and C should be an interface.
- We can’t have more than one class in multiple bounds.

Practice Programs

```java
package genericpackage;

class GenericTypes<T> {
    private T t;

    public T get() {
        return this.t;
    }

    public void set(T t1) {
        this.t = t1;
    }
}
```

```java
public class GenericsMethodsBounds {

    //Generics in method
    public static <T> boolean checkEquals(GenericTypes<T> g1, GenericTypes<T> g2){
        return g1.get().equals(g2.get());
    }


    //bounded type parameter
    // <T extends A & B & C>. In this case, A can be an interface or class.
    //If A is class then B and C should be an interface
    public static <T extends Comparable<T>> int compare(T t1, T t2){
        return t1.compareTo(t2);
    }


```

```java


    public static void main(String args[]){
        GenericTypes<String> g1 = new GenericTypes<>();
        g1.set("Bini");

        GenericTypes<String> g2 = new GenericTypes<>();
        g2.set("bini");

       boolean check = GenericsMethodsBounds.<String>checkEquals(g1, g2);
          System.out.println(check);

        //above statement can be written simply as
          check = GenericsMethodsBounds.checkEquals(g1,g2);
          System.out.println(check);

          System.out.println(GenericsMethodsBounds.compare("bini","bini")); // if equals =0
          System.out.println(GenericsMethodsBounds.compare("bini","Bini")); // lexicographically greater +
          System.out.println(GenericsMethodsBounds.compare("bini","anu")); //lexicographically
    }
}
```

5. Wildcard in Java

- Wildcards in Java are basically the question marks (?) which we use in generic programming
- It basically represents the unknown type.
- We use Java Wildcard widely in situations such as in a type of parameter, local variable, or field and also as a return type.

```java
package com;

import java.util.ArrayList;
import java.util.List;

public class UnboundedWildcard {

    private static void printList(List < ?>list) {
            System.out.println(list);
        }

        public static void main(String[] args) {
            //Integer List
              List < Integer > intList = new ArrayList();
                intList.add(10);
                intList.add(20);

                  printList(intList);

            //Double list
                  List < Double > doubleList = new ArrayList();
                    doubleList.add(13.2);
                    doubleList.add(15.6);

            printList(doubleList);
        }

      }
```

Annotation

Annotations are used to provide supplemental information about a program.

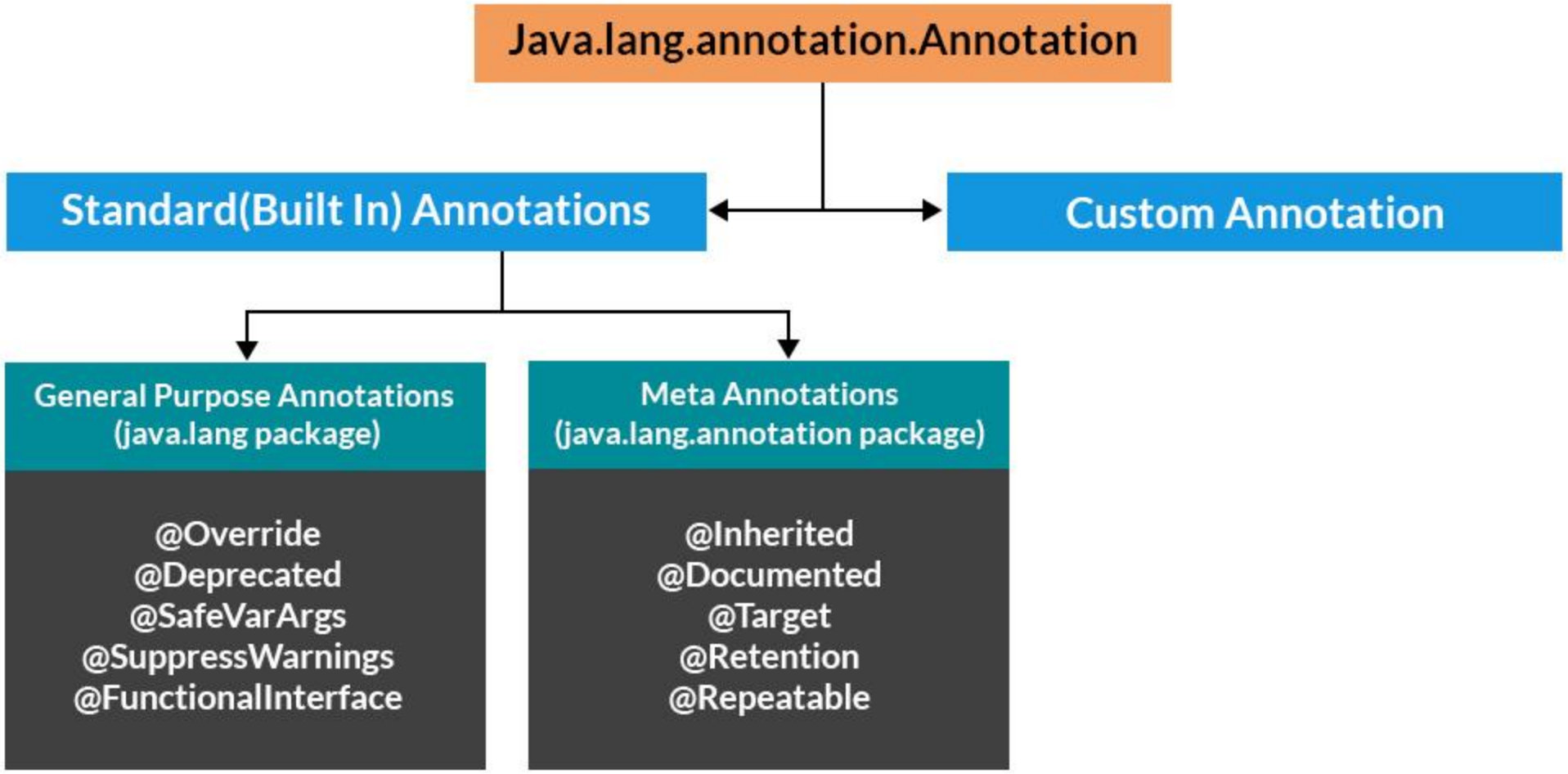
1. Annotations start with '@'.
2. Annotations do not change the action of a compiled program.
3. Annotations help to associate metadata (information) to the program elements i.e. instance variables, constructors, methods, classes, etc.
4. Annotation Placement: You can place Java annotations above classes, interfaces, methods, method parameters, fields and local variables.
5. Annotations are not pure comments as they can change the way a program is treated by the compiler.

Java Annotation Purposes

Java annotations are typically used for the following purposes:

- Compiler instructions
- Build-time instructions **
- Runtime instructions

**Java annotations can be be used at build-time, when you build your software project. The build process includes generating source code, compiling the source, generating XML files (e.g. deployment descriptors), packaging the compiled code and files into a JAR file etc. Build tools(an automatic build tool like Apache Ant or Apache Maven) may scan your Java code for specific annotations and generate source code or other files based on these annotations.


Java.lang.annotation.Annotation
Standard(Built In) Annotations
Custom Annotation
General Purpose Annotations
(java.lang package)
@Override
@Deprecated
@SafeVarArgs
@SuppressWarnings
@FunctionalInterface
Meta Annotations
(java.lang.annotation package)
@Inherited
@Documented
@Target
@Retention
@Repeatable

Built-in Java Annotations

Java comes with three built-in annotations which are used to give the Java compiler instructions. These annotations are:

@Deprecated

@Override

@SuppressWarnings

@Deprecated

The @Deprecated annotation is used to mark a class, method or field as deprecated(above the class,method or field), meaning it should no longer be used. If your code uses deprecated classes, methods or fields, the compiler will give you a warning.

```
@Deprecated
public class MyComponent {
}
```

Ie. MyComponent class as deprecated.

When you use the @Deprecated annotation, it is a good idea to also use the corresponding @deprecated JavaDoc symbol, and explain why the class, method or field is deprecated, and what the programmer should use instead.

```
@Deprecated
/**
 @deprecated Use MyNewComponent instead.
*/
```

@Override

The @Override Java annotation is used above methods that override methods in a superclass. If the method does not match a method in the superclass, the compiler will give you an error.

The @Override annotation is not necessary in order to override a method in a superclass. It is a good idea to use it still, though. In case someone changed the name of the overridden method in the superclass, your subclass method would no longer override it. Without the @Override annotation you would not find out. With the @Override annotation the compiler would tell you that the method in the subclass is not overriding any method in the superclass.

@Override -> Compiler Warning Demo

Child.java

```java
package annotationpackage;


// Program to show that Annotations are Not Barely Comments

class Parent {

    // Method
    public void display(){
        System.out.println("Base display()");
    }
}
public class Child extends Parent{

    // Overriding method as already up in above class
    @Override
    public void display(int x){
        // Print statement when this method is called
        System.out.println("Child class display(int )");
    }

    /*
     * If you remove parameter (int x) or we remove @override, the program compiles fine.
     */

```

```java


    // Main method
    public static void main(String args[]){
        // Creating object of this class inside main()
        Child obj = new Child();

        // Calling display() method inside main()
        obj.display();
    }
}
```

@SuppressWarnings

The @SuppressWarnings annotation makes the compiler suppress warnings for a given method. For instance, if a method calls a deprecated method, or makes an insecure type cast, the compiler may generate a warning. You can suppress these warnings by annotating the method containing the code with the @SuppressWarnings annotation.

Here is a @SuppressWarnings Java annotation example:

```
@SuppressWarnings
public void methodWithWarning() {
}
```

What Are Autoboxing and Unboxing?

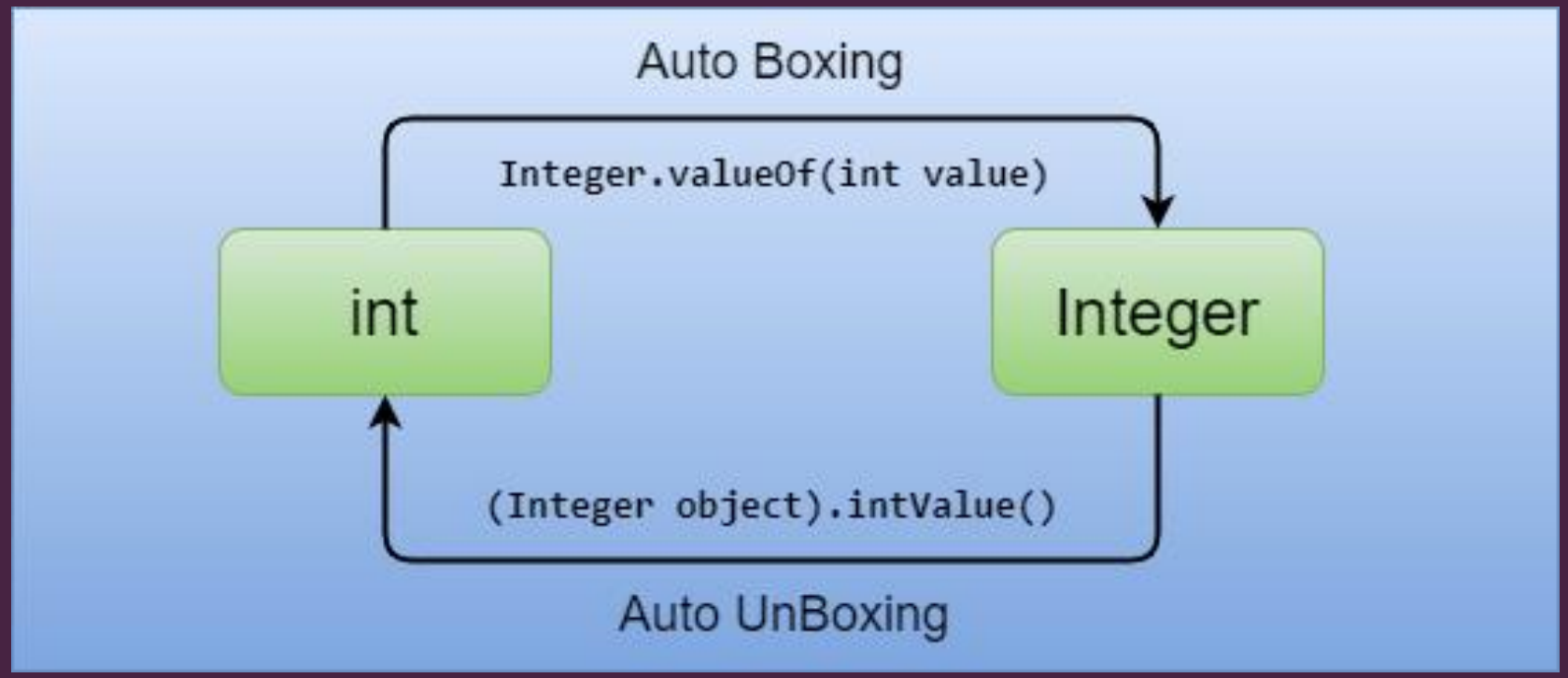
Autoboxing is the automatic conversion that the Java compiler makes between the primitive types and their corresponding object wrapper classes.

E.g.converting an int to an Integer, a double to a Double, and so on.

If the conversion goes the other way, this is called unboxing.

In Java 1.5, Java added concept of autoboxing and unboxing that deal with conversion of primitive type to object and vice versa.

How the compiler converts primitive to the wrapper and vice versa



While working with Java you might have seen that a lot of things Java Compiler does internally. It becomes easier for the programmer to write clean code but at the same time, the developer should know the internal working of the functionality which he/she is writing in the application.

Autoboxing and unboxing lets developers write cleaner code, making it easier to read. The following table lists the primitive types and their corresponding wrapper classes, which are used by the Java compiler for autoboxing and unboxing:

Primitive type	Wrapper class
boolean	Boolean
byte	Byte
char	Character
float	Float
int	Integer
long	Long
short	Short
double	Double

Whenever we use object of Wrapper class in an expression, autoboxing is done by JVM

PrimitiveWrapper.java

```java
package autoboxunboxpackage;

public class PrimitiveWrapper {
    public static void main(String[] args) {
        Integer i = 100;
        // Auto-boxing of int i.e converting primitive data type int to a Wrapper class Integer
        System.out.println(i);

        Character ch = 'a';
        System.out.println(ch);

        Byte b = 12;
        System.out.println(b);
    }
}
```

ArrayListAutoBox.java

```java
package autoboxunboxpackage;

import java.util.ArrayList;

public class ArrayListAutoBox {
    public static void main(String[] args){
        ArrayList<Integer> arrayList = new ArrayList<Integer>();
        arrayList.add(100); // autoboxing int to Integer
        arrayList.add(200);
        arrayList.add(300);
        for(Integer i : arrayList) {
            System.out.println(i);
        }
    }

}
```

MethodAutoUnBox.java

```java
package autoboxunboxpackage;

public class MethodAutoUnBox {
    // autoboxing in parameters
        static int add(Integer a, Integer b) {
            // unboxing in return
            return a+b;
        }

        public static void main(String args[]) {

            int sum = add(10,20);

            System.out.println("sum = "+sum);

            }

}
```

Enum(Enumerations)

An enum is a special "class" that represents a group of constants (unchangeable variables, like final variables).

To create an enum, use the enum keyword (instead of class or interface), and separate the constants with a comma. Note that they should be in uppercase letters:

EnumOutSide.java

```java
package enumpackage;

enum Car{

    MARUTI,HONDA,TESLA;

}

public class EnumOutSide{

    public static void main(String [] args) {

        Car myCar= Car.HONDA;

        System.out.println(myCar);

    }

}
```

```java
package enumpackage;

public class EnumInside {
        enum Diet {
            BREAKFAST,
            LUNCH,
            DINNER
        }

        public static void main(String[] args) {
            Diet myVar = Diet.LUNCH;
            System.out.println(myVar);
        }
    }
```

EnumSwitchDemo.java

```java
package enumpackage;


enum Size {
     SMALL, MEDIUM, LARGE, EXTRALARGE
    }

    class Test {
     Size pizzaSize;

     public Test(Size pizzaSize) {
       this.pizzaSize = pizzaSize;
     }

     public void orderPizza() {
       switch(pizzaSize) {
         case SMALL:
           System.out.println("I ordered a small size pizza.");
           break;
         case MEDIUM:
           System.out.println("I ordered a medium size pizza.");
           break;
         default:
           System.out.println("I don't know which one to order.");
           break;
       }
     }
    }
```

```
class EnumSwitchDemo {
 public static void main(String[] args) {
   Test t1 = new Test(Size.MEDIUM);
   t1.orderPizza();
 }
}
```

Practice Programs

```java
package enumpackage;

import java.util.Scanner;

enum Account {
    SAVINGS, CURRENT, CREDIT
}

class AccountType {
    Account type;

    public AccountType(Account type) {
        this.type = type;
    }

    public void accountCoverage() {
        switch (type) {
            case SAVINGS:
                System.out.println("You have to maintain Rs.1000/- balance in your account");
                break;
```

```
            case CURRENT:
                System.out.println("You have to maintain Rs.15000/- balance in your
                account");
                break;
            case CREDIT:
                System.out.println("minimum balance is not applicable");
                break;
            default:
                System.out.println("I don't know ");
                break;
        }
    }

}
```

```java
public class EnumProgram {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        AccountType aType;
        char choice;

        do {
            System.out.println("ACCOUNT
            TYPE\n=====================================");

            System.out.println("1. SAVINGS");
            System.out.println("2. CURRENT");
            System.out.println("3. CREDIT");
            System.out.println("4. EXIT");
            System.out.println("what is your account type? 1 or 2 or
            3..?");
            int ac = sc.nextInt();

            switch (ac) {
                case 1:
                    aType = new AccountType(Account.SAVINGS);
                    aType.accountCoverage();
                    break;
```

```
        case 2:
            aType = new AccountType(Account.CURRENT);
            aType.accountCoverage();
            break;
        case 3:
            aType = new AccountType(Account.CREDIT);
            aType.accountCoverage();
            break;
        case 4:
            System.exit(0);
        default:
            System.out.println("invalid choice");
            break;
        }
        System.out.println("Do you want to continue?y/n");
        choice = sc.next().charAt(0);
        if (choice == 'y' || choice == 'Y’)
            continue;
        else
            break;

    } while (true);
}
}
```

Varargs (Variable Arguments)

Allows a method to accept a variable number of arguments.

```java
public class VarargsExample {
    public static void printNumbers(int... numbers) {
        for (int num : numbers) {
            System.out.print(num + " ");
        }
        System.out.println();
    }

    public static void main(String[] args) {
        printNumbers(1, 2, 3);
        printNumbers(10, 20, 30, 40, 50);
    }
}
```

```
public class VarargsMainExample {
    public static void main(String... args) {  // Using varargs instead of String[]
        for (String arg : args) {
            System.out.println(arg);
        }
    }
}
```

Static Imports

Allows you to import static members (methods and variables) directly, reducing redundant code.

```java
import static java.lang.Math.*;  // Import static members

public class StaticImportExample {
    public static void main(String[] args) {
        System.out.println(sqrt(25));  // Instead of Math.sqrt(25)
        System.out.println(PI);        // Instead of Math.PI
    }
}
```

Concurrency Utilities (java.util.concurrent package)

Introduced new multithreading utilities like Executor, Callable, Future, and ConcurrentHashMap.

Thank you ☺ Happy learning ☺